

**Multiple object recognition:**

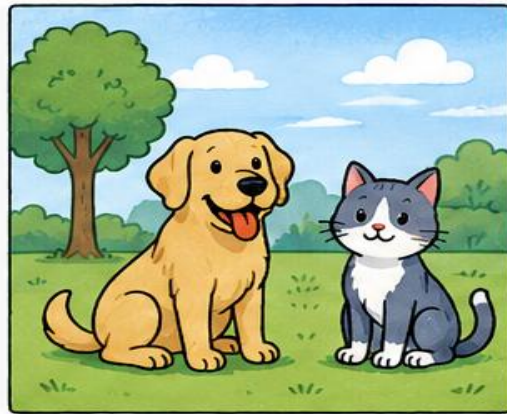
**You Look Only Once  
YOLO**

*Vladimir J N Bykov*

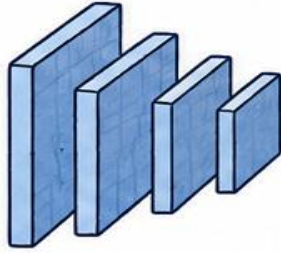
# Hur fungerar YOLO?

## Jämförelse: CNN vs YOLO

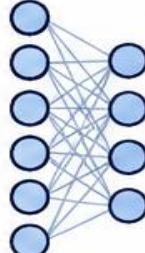
### CNN – Bildklassificering “Vad finns i bilden?”



1. CNN extraherar features



2. Fullt kopplade lager



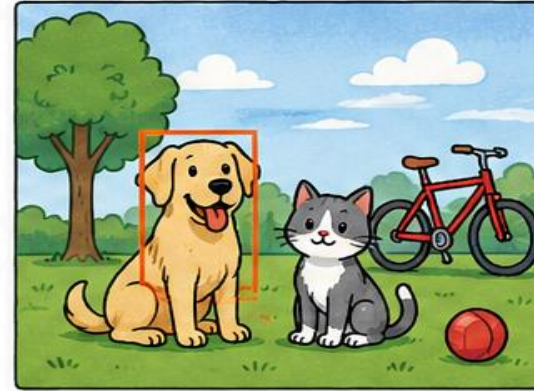
3. Output: en klass

Hund: 98%  
Katt: 2%  
Bil: 0%  
Fågel: 0%  
...

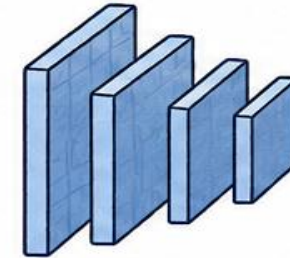


CNN ger en **enda etikett** för hela bilden.

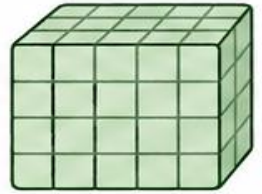
### YOLO – Objektigenkänning “Vad finns i bilden och var finns det?”



1. CNN extraherar features



2. YOLO-head



3. Predikterar flera objekt samtidigt  
För varje objekt: klass + position + storlek + confidence

|                                                                                       |       |                                                                                       |                                        |
|---------------------------------------------------------------------------------------|-------|---------------------------------------------------------------------------------------|----------------------------------------|
|    | Hund  |    | x: 120 y: 200 w: 120 h: 160 conf: 0.98 |
|   | Katt  |   | x: 300 y: 210 w: 100 h: 130 conf: 0.95 |
|  | Cykel |  | x: 480 y: 190 w: 150 h: 100 conf: 0.90 |
|  | Boll  |  | x: 530 y: 300 w: 40 h: 40 conf: 0.85   |



YOLO tittar på hela bilden på **en gång** och hittar flera objekt med klass och position i en **enda genomgång** – därför är det mycket snabbt!

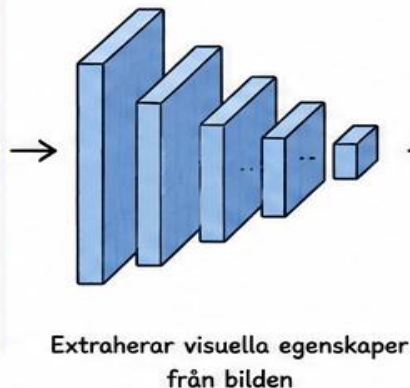


# YOLO: Vad är Detection Head och vad gör den?

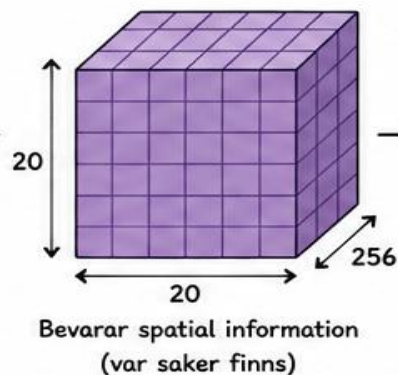
## 1. Indata: Bild



## 2. Backbone CNN (t.ex. Darknet/CSP)

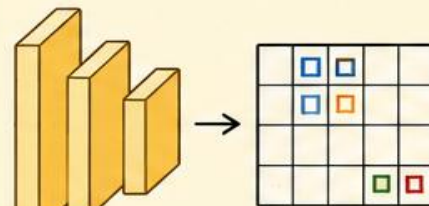


## 3. Feature Maps (exempel: 20 x 20 x 256)



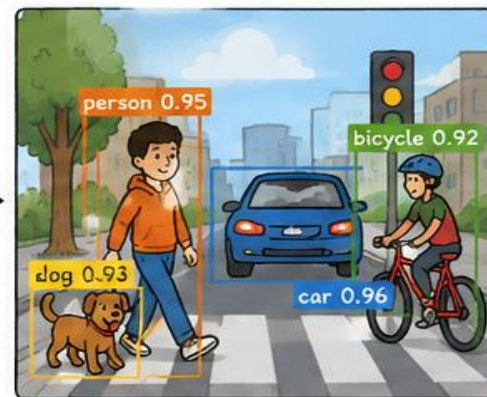
## 4. DETECTION HEAD

Tar feature maps och förutsäger objekt för varje område på kartan



Består av konvolutionslager som gör prediktioner utan Flatten eller Dense-lager

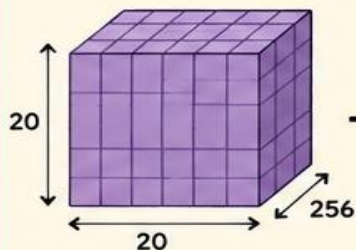
## 5. Slutliga prediktioner



Klass + position (bbox) + confidence

## ZOOM: Vad gör Detection Head?

Input: Feature Maps  
(20 x 20 x 256)



Varje cell "ser" ett område i originalbilden.

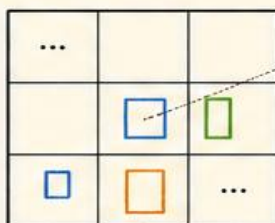
### Steg 1: Convolution

Flera konvolutionslager för att förstå innehållet i varje område.



### Steg 2: Prediktion per cell

För varje cell och varje anchor box (fördefinierade storlekar/former) görs prediktioner.

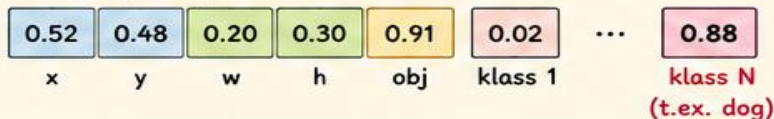


### Steg 3: Output per prediction

För varje anchor box predikteras:

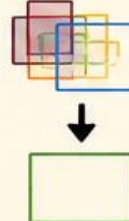


Exempel på output för en anchor box i en cell:



### Steg 4: Filtrering (NMS)

Ta bort överlappande boxar med lägre confidence.



Kvar blir de bästa boxarna för varje objekt.



**Sammanfattning:** Detection Head är YOLO:s "beslutsdel" som tittar på feature maps och, för varje område på bilden, predikterar om det finns ett objekt, vad det är, var det är och hur stort det är – allt i en enda genomgång av bilden!

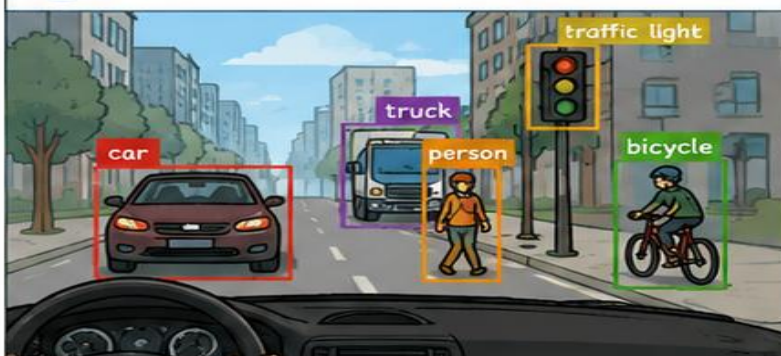




# VAR ANVÄNDS YOLO?

YOLO hittar och lokaliserar objekt i bilder och video – i realtid!

## 1 SJÄLVKÖRANDE BILAR



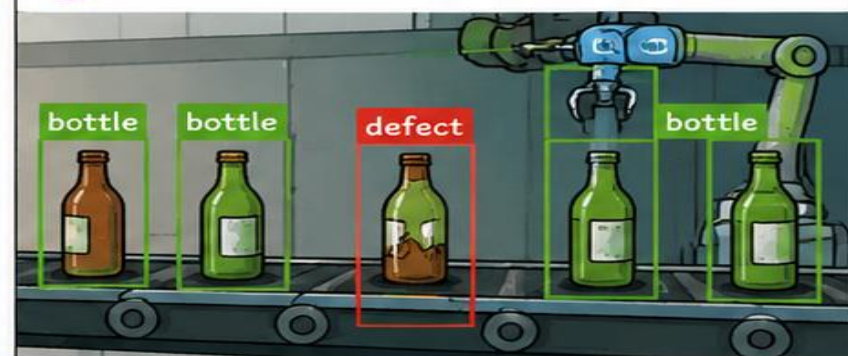
Upptäcker fordon, fotgängare, cyklister, trafikskyltar och hinder för säker körning.

## 2 ÖVERVAKNING & SÄKERHET



Övervakar områden, räknar personer, upptäcker avvikande beteenden eller objekt.

## 3 INDUSTRI & KVALITETSKONTROLL



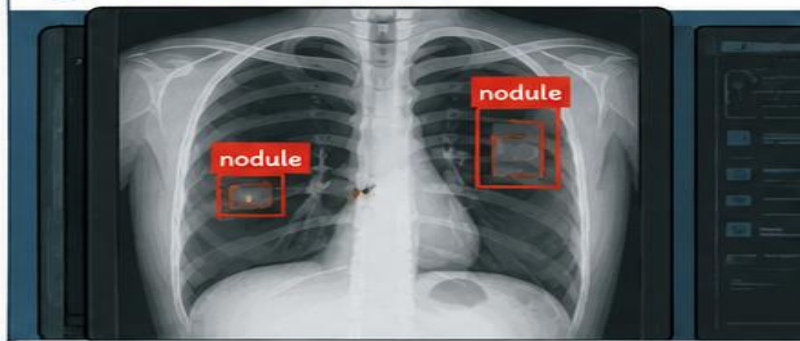
Identifierar defekta produkter, saknade delar och säkerställer kvalitet i produktionslinjer.

## 4 BUTIK & DETALJHANDEL



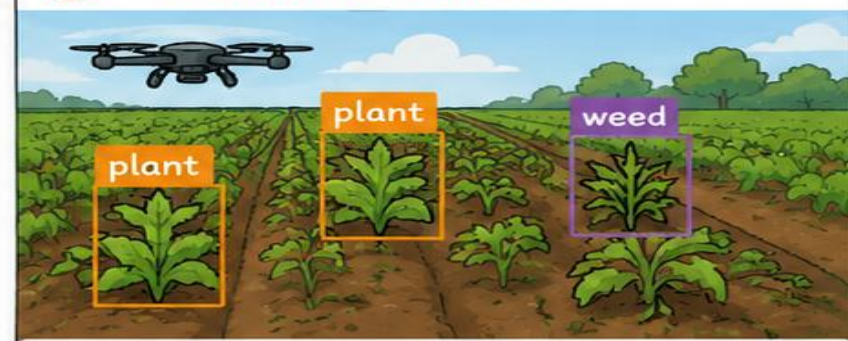
Inventering i hyllor, automatisk kassa, kundflödesanalys och produktigenkänning.

## 5 MEDICIN & HÄLSA



Upptäcker tumörer, frakturer, celler och andra avvikelser i röntgen, CT, MR och mikroskopibilder.

## 6 JORDBRUK & MILJÖ



Övervakar grödor, upptäcker ogräs, räknar djur och skyddar skogar och natur.

VARFÖR  
YOLO?



**Snabbt**  
Realtidsdetektion



**Exakt**  
Hög noggrannhet



**Mångsidigt**  
Fungerar för många  
typer av objekt



**Effektivt**  
Kan köras på allt från  
moln till edge-enheter

YOLO ser världen  
och förstår den –  
direkt!



# Yolo funkar på CPU, utan TF

```
from ultralytics import YOLO
import cv2
import matplotlib.pyplot as plt

model = YOLO("yolo11n.pt")  # liten, snabb pretrained modell

results = model("bus_people.jpg")

annotated = results[0].plot()

plt.figure(figsize=(10, 8))
plt.imshow(cv2.cvtColor(annotated, cv2.COLOR_BGR2RGB))
plt.axis("off")
plt.show()
```



# YOLO





# YOLO



(a) Bathroom scene image



(b) Bedroom scene image



(c) Dining room scene image



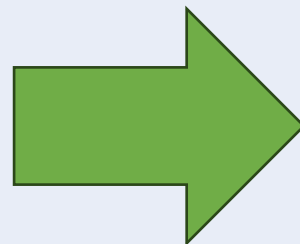
(d) Kitchen scene image



(e) Living room scene image



(f) Office scene image



(a) Bathroom scene image



(b) Bedroom scene image



(c) Dining room scene image



(d) Kitchen scene image



(e) Living room scene image



(f) Office scene image

*Tack för er tid*